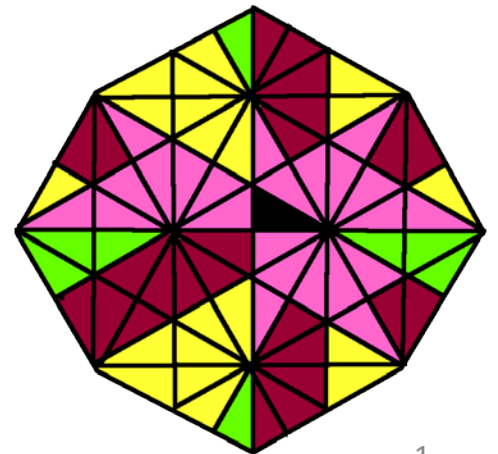


Structuri de Date

Anul universitar 2019-2020

Prof. Adina Magda Florea



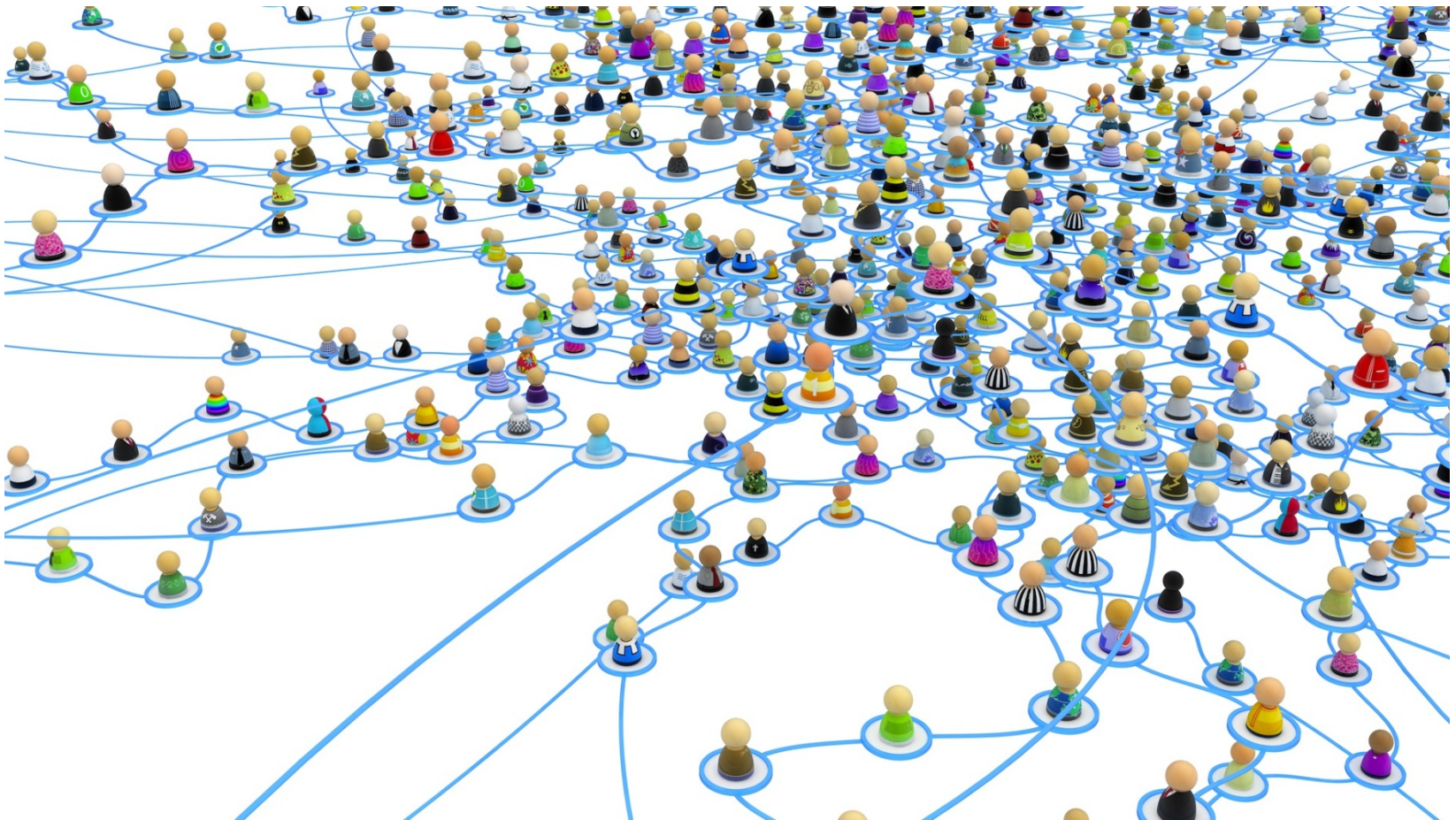
Curs Nr. 9

Grafuri

- ☐ Exemple
- ☐ Noțiuni de bază
- ☐ Probleme asociate grafurilor
- ☐ ADT Graf
- ☐ Reprezentarea grafurilor
- ☐ Parcurgerea grafurilor
- ☐ Componente conexe

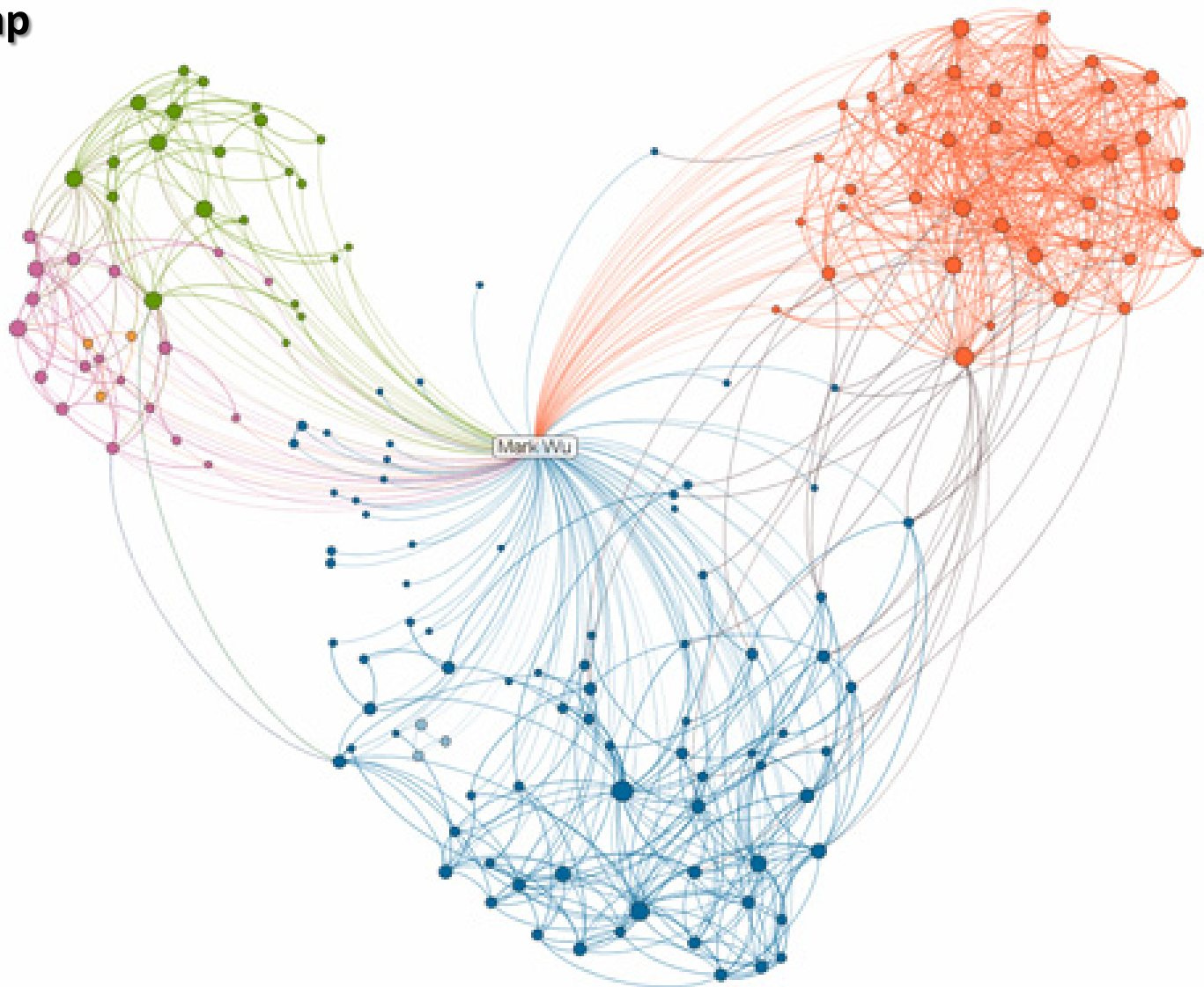
1. Exemple de grafuri

Rețele sociale



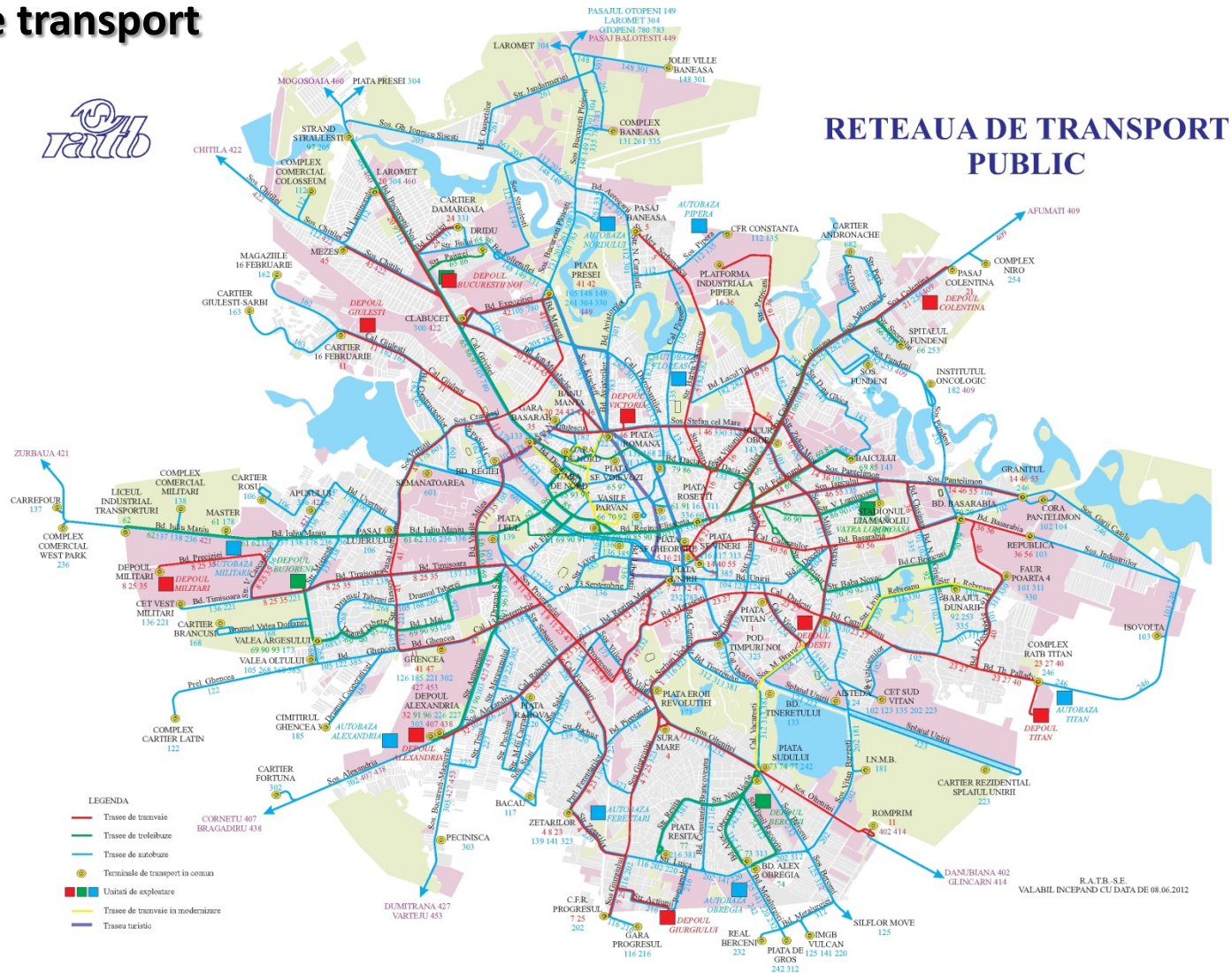
Exemple de grafuri

LinkedIn Map



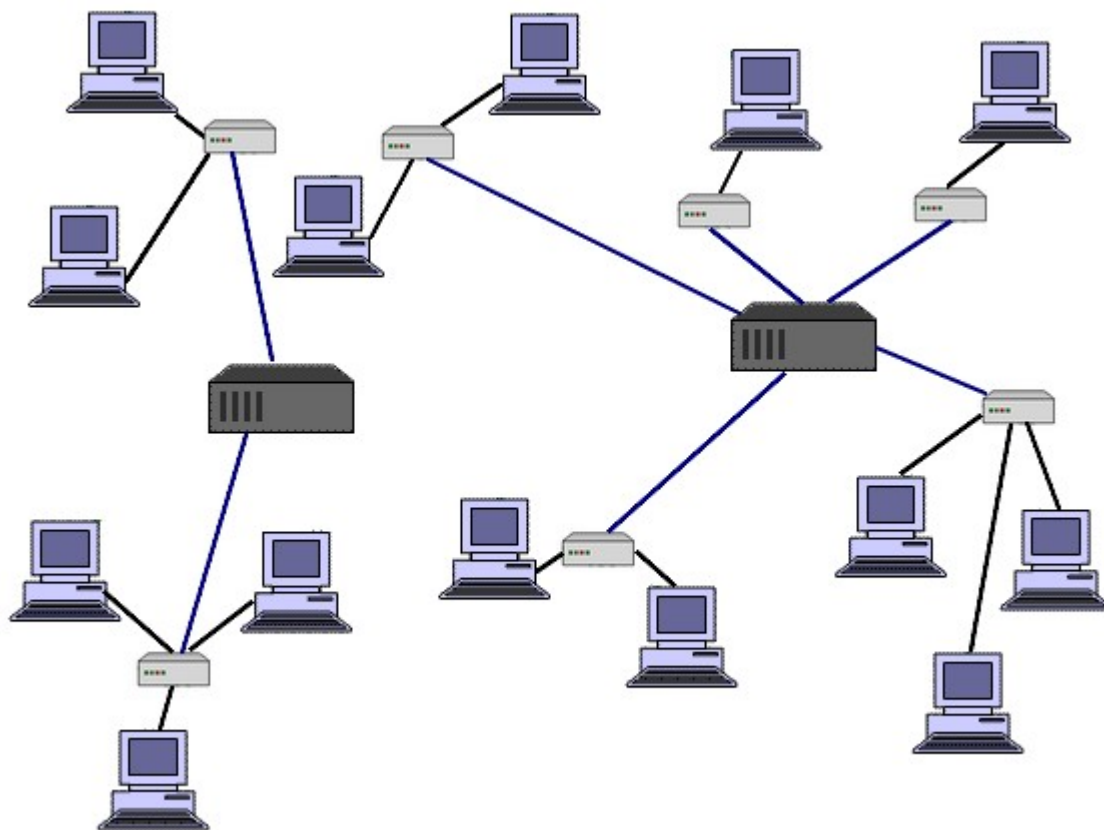
Exemple de grafuri

Rețele de transport



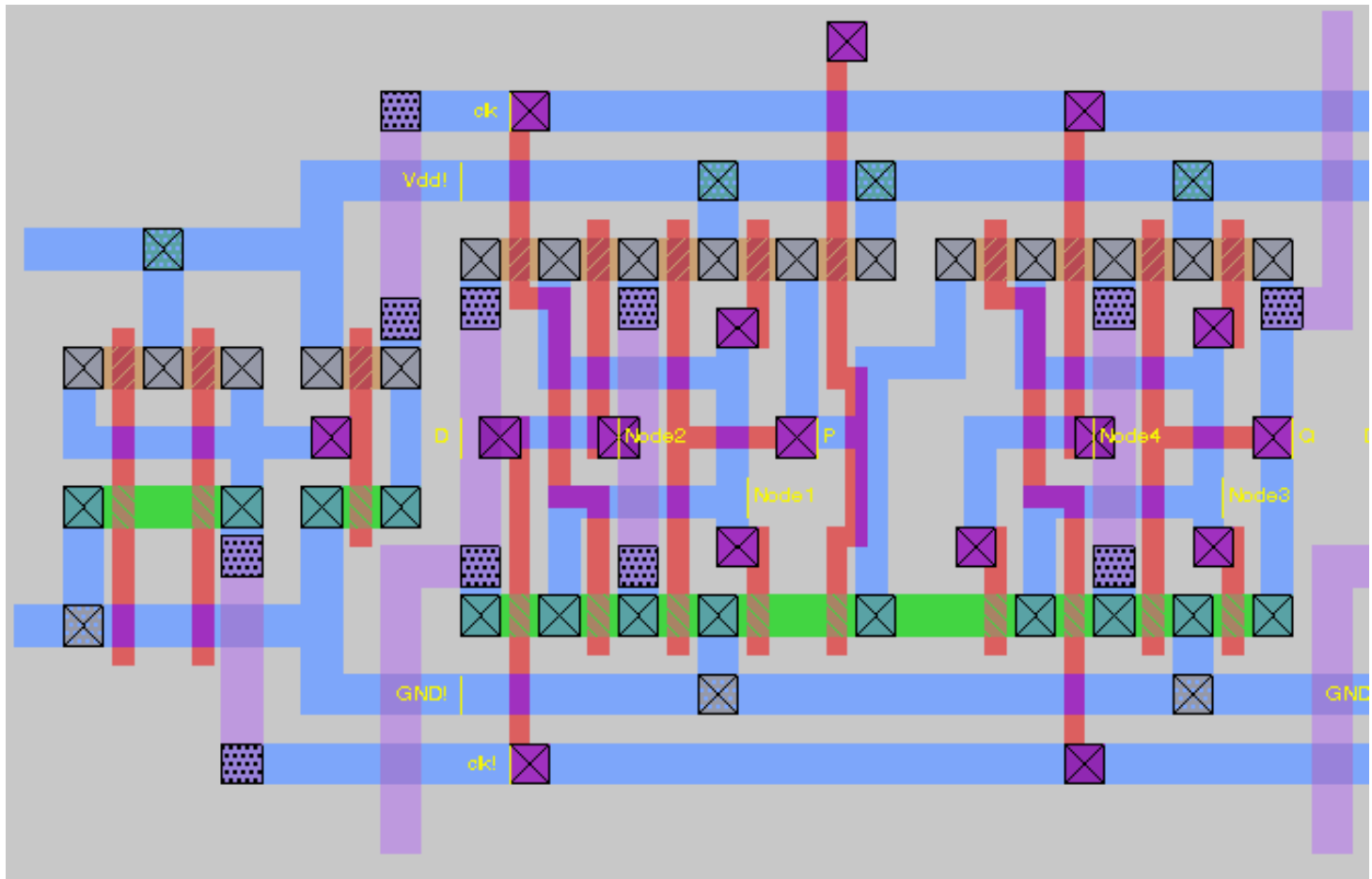
Exemple de grafuri

Rețele de calculatoare



Exemple de grafuri

VLSI layout



2. Noțiuni de bază

- Un graf este o modalitate de a reprezenta **relații între obiecte**.
- Un **graf** se definește printr-o mulțime de **noduri** sau **vârfuri** (*vertices*) și o mulțime de legături între aceste noduri numite **arce** sau **muchii** (*edges*).

$G = (V, E)$, V = mulțimea de noduri

E este o **relație binară** peste V

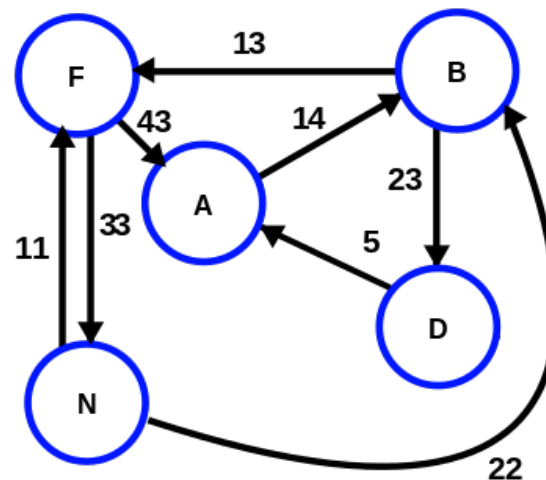
$E \subseteq V \times V$, $E = \{(u,v) \mid u,v \in V\}$

- **Grafuri neorientate** (undirected)
- **Grafuri orientate** (directed sau digraph)

Noțiuni de bază

Grafuri ponderate (*weighted graphs*)

- Au asociat un cost arcelor
- Grafuri neorientate ponderate
- Grafuri orientate ponderate
- De obicei costul este pozitiv



Noțiuni de bază

- **Capetele unui arc (endpoints)** – cele 2 noduri legate de un arc (u,v)
- Două noduri u , v sunt **adiacente** dacă există un arc (u,v)
- Un **arc (u,v)** se spune că este **incident** într-un nod dacă nodul este u sau v
- Dacă graful este **orientat**, u – **originea arcului** (u,v) , v – **destinația arcului** (u,v)
- **Arcuri de ieșire** (outgoing) ale unui **nod v** – arcele orientate care au ca origine acel nod v
- **Arcuri de intrare** (incoming) ale unui **nod v** – arcele orientate care au ca destinație acel nod v

Noțiuni de bază

- **Gradul unui nod** v este numărul de arce incidente cu acesta – **$\deg(v)$**
 - Nod izolat – $\deg(v) = 0$
 - **Gradul de intrare al unui nod** dintr-un graf orientat **$\text{indeg}(v)$** – numărul arcelor care intră în acel nod. 
 - **Gradul de ieșire al unui nod** dintr-un graf orientat **$\text{outdeg}(v)$** – numărul arcelor care ies în acel nod. 
- **Ordinul unui graf** este numărul de noduri din graf
 $n = |V|$
 - **Dimensiunea unui graf** este egală cu numărul de muchii $m = |E|$

Noțiuni de bază

- O **cale (drum)** de lungime k între două noduri a și b este o succesiune de noduri $v_0, v_1, \dots, v_k$, cu $v_0=a$ și $v_k=b$ și $\{v_{i-1}, v_i\} \in E$ pentru $i=1,k$.
- Dacă **există o cale de la a la b** , atunci **b este accesibil din a** .
- O **cale simplă** are toate nodurile distincte.
- Un **ciclu** este un drum $(v_0, v_1, \dots, v_k)$ în care $v_0 = v_k$.
- O **bucă** (a,a) este un ciclu de lungime 1.
- Un **ciclu simplu** are toate nodurile distincte (exceptând nodul de plecare) și nu conține bucle.
- Un **graf aciclic** (pădure) **nu conține cicluri**.

Noțiuni de bază

- **Graf conex** (connected graph) - un graf neorientat pentru care oricare două vârfuri pot fi unite printr-un drum
- **Componentele conexe** ale unui graf neorientat sunt clasele de echivalență ale vârfurilor prin relația “este accesibil din”.
- Un graf neorientat este conex, dacă are o singură componentă conexă.
- Un graf conex aciclic este arbore liber.
- Graf dens $E \sim O(|V|^2)$
- Graf rar $E \sim O(|V|)$

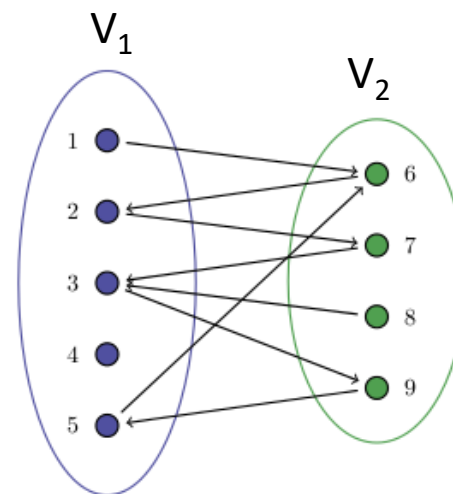
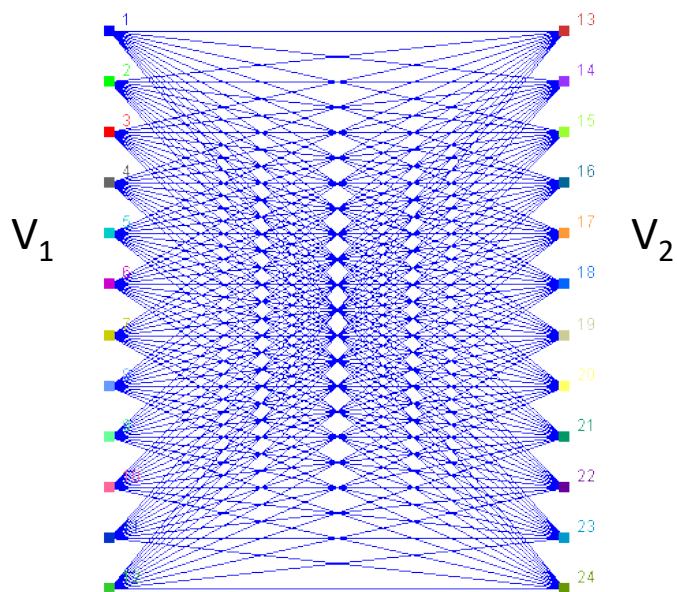
Noțiuni de bază

- **Graf tare conex** (strongly connected graph) – un graf orientat cu proprietatea: pentru orice două vârfuri u și v , u este accesibil din v și v este accesibil din u
- Graful $G' = (V', E') \subseteq G$ este **subgraf al grafului** $G = (V, E)$, dacă $V' \subseteq V$ și $E' \subseteq E$
- Graful $G' = (V, E')$ este un **subgraf de acoperire** (spanning subgraph) al grafului $G = (V, E)$ dacă $E' \subseteq E$
- Un **arbore de acoperire** (spanning tree) $G' = (V, E')$ al unui graf $G = (V, E)$ este un subgraf de acoperire care este un arbore liber



Noțiuni de bază

- Un **graf bipartit** este un graf în care mulțimea nodurilor V se partiționează
 - $V = V_1 \cup V_2$, $V_1 \cap V_2 = \emptyset$ astfel încât
 - pentru $\forall (u, v) \in E$ $u \in V_1$ și $v \in V_2$
sau $u \in V_2$ și $v \in V_1$





Proprietăți

- **P1:** Fie un *graf neorientat* G cu m arce și mulțimea de vârfuri V

$$\sum_{v \in V} \deg(v) = 2m$$

- **P2:** Fie un *graf orientat* G cu m arce și mulțimea de vârfuri V

$$\sum_{v \in V} \text{indeg}(v) = \sum_{v \in V} \text{outdeg}(v) = m$$



Proprietăți

- **P3:** Fie un **graf G simplu** cu **n** noduri și **m** arce.
 - Dacă G este **neorientat** atunci **$m \leq n(n-1)/2$**
 - Dacă G este **orientat** atunci **$m \leq n(n-1)$** **$O(n^2)$**
- **P4:** Fie un **graf G** cu **n** noduri și **m** arce.
 - Dacă G este conex atunci $m \geq n-1$
 - Dacă G este arbore atunci $m = n-1$
 - Dacă G este o pădure atunci $m \leq n-1$

3. Probleme asociate grafurilor

Graf neorientat

- Este graful conex?
- Care sunt componentele conexe ale grafului?
- Graful conține cicluri?

Graf orientat

- Este graful tare conex?
- Care sunt componentele tare conexe?
- Graful conține cicluri?
- **Graf orientat aciclic** (DAG = Directed Acyclic Graph); este graful un DAG?

Probleme asociate grafurilor

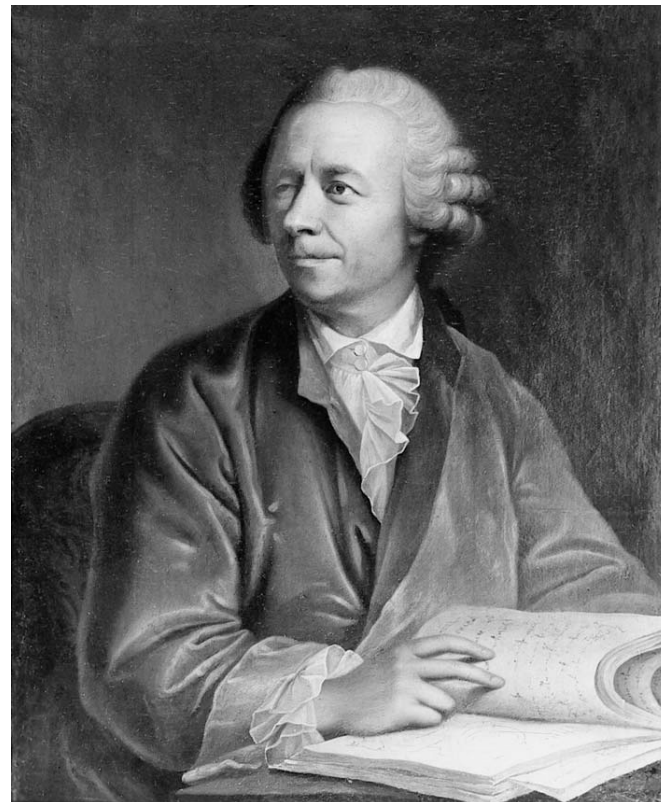
Cale Euleriană – o cale care vizitează fiecare arc din G exact o dată

Ciclu Eulerian – un ciclu care vizitează fiecare arc din G exact o dată

Leonhard Euler

1707, Basel, Elveția

- 1783

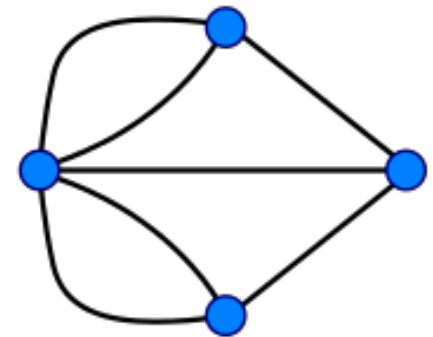
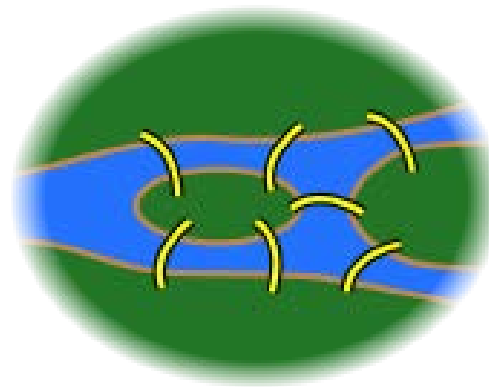
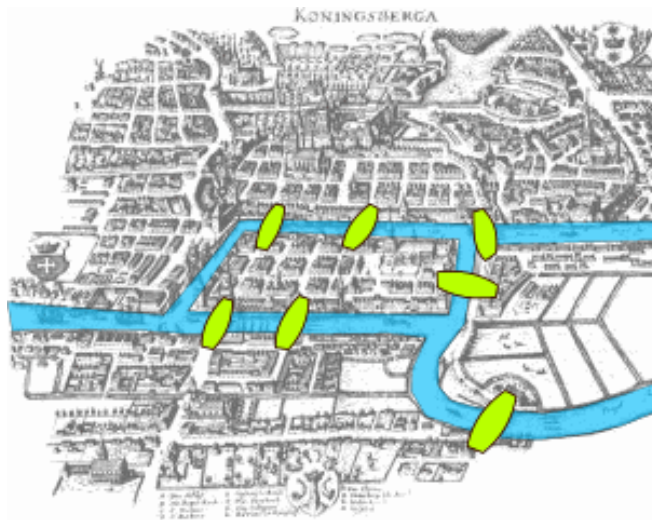


Probleme asociate grafurilor

Leonhard Euler

Cele 7 poduri din Königsberg Prusia peste râul Pregel

Au o cale Euleriana? Au un ciclu Eulerian?



Probleme asociate grafurilor

Cale Hamiltoniană – este o cale simplă într-un graf (orientat sau neorientat)

Ciclu Hamiltonian – ciclu simplu în care se vizitează fiecare nod din G exact o dată

William Rowan Hamilton

1805, Dublin, Ireland

- 1865



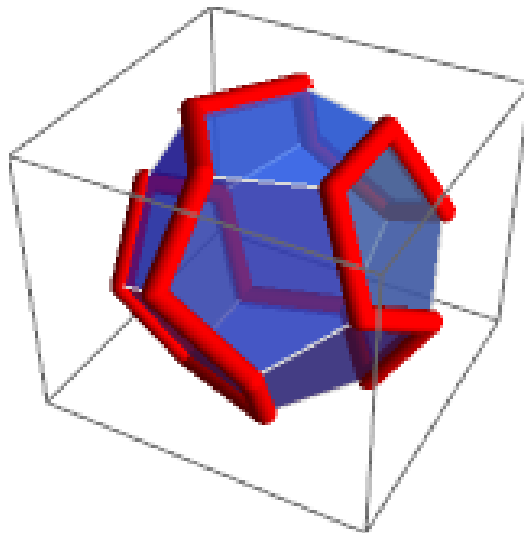
Probleme asociate grafurilor

Ciclu Hamiltonian – ciclu simplu în care se vizitează fiecare nod din G exact o dată

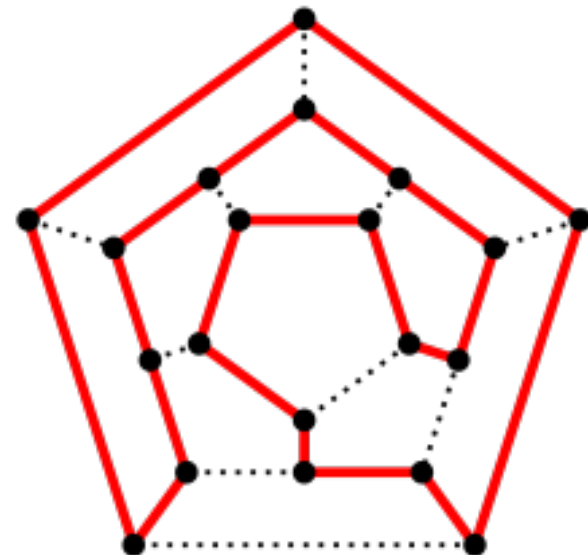
William Rowan Hamilton

1805, Dublin, Ireland

- 1865



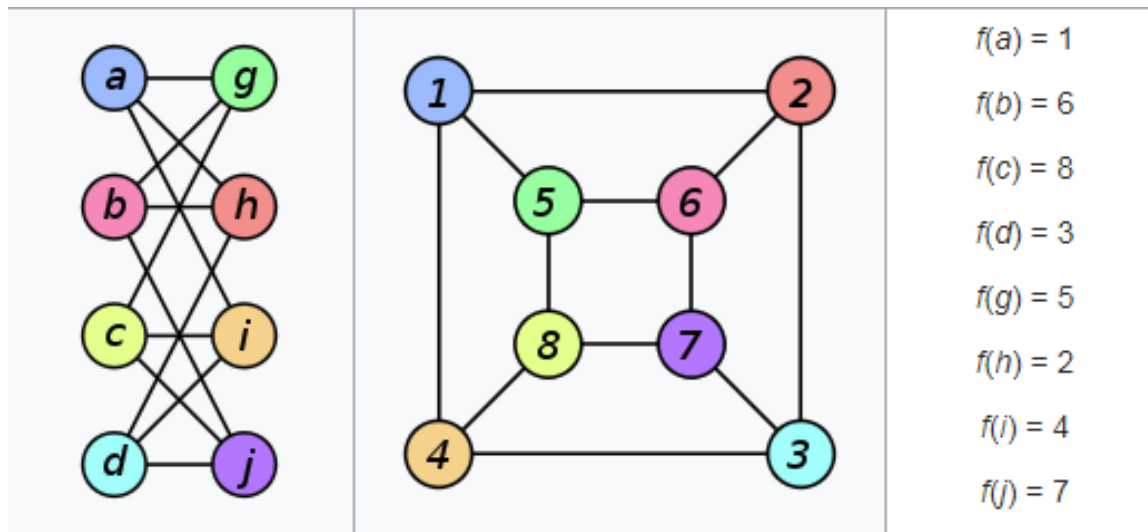
Jocul icosian



Probleme asociate grafurilor

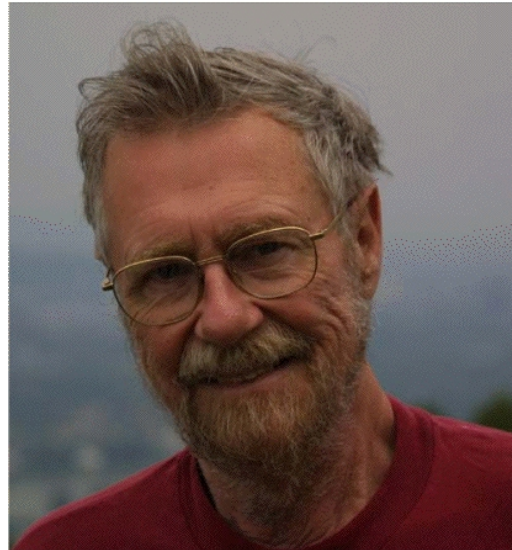
- Parcurgerea grafurilor – orientate / neorientate
- Izomorfismul grafurilor

$f: V(G1) \rightarrow V(G2)$



Probleme asociate grafurilor

- **Calea cea mai scurtă** între două noduri din graf pentru un **graf ponderat**
 - Algoritmul lui Edsger Wybe Dijkstra
1930 - 2002 Rotterdam, Olanda



Probleme asociate grafurilor

- Arborele de acoperire de cost minim – graf ponderat
 - Algoritmul lui Prim
 - Vojtěch Jarník 1897-1970, matematician ceh
 - Robert Clay Prim, 1921- cs american
 - Algoritmul lui Kruskal
 - Joseph Bernard Kruskal, Jr., 1928-2010, matematician și cs american
- Problema comis voiajorului – graf ponderat



4. ADT Graf

- **initGraph** – inițializare graf
- **numV** – întoarce numărul de noduri din graf
- **numE** – întoarce numărul de arce din graf
- **getEdge(u,v)** – întoarce arcul dintre **u** și **v** dacă există, altfel nul
- **endVertices(e)** – întoarce un vector ce conține cele 2 noduri care definesc pe **e**. Dacă graful este orientat, primul vârf este originea, cel de al doilea destinația

ADT Graf

- **deg(v)** – întoarce gradul nodului **v**
- **indeg(v)** - întoarce gradul de intrare a nodului **v**
- **outdeg(v)** – întoarce gradul de ieșire a nodului **v**
- **insertVertex(x)** – crează și întoarce un nou nod ce memorează elementul **x**
- **insertEdge(u,v,x)** – crează și întoarce un nou arc de la nodul **v** la nodul **u** și îl etichetează cu **x**; eroare dacă arcul există deja
- **removeVertex(v)** – elimină nodul **v** și toate arcele incidente lui
- **removeEdge(e)** - elimină arcul **e** din graf

5. Reprezentarea grafurilor

- **Număr noduri / vârfuri V , număr arce E**
- **Se realizează o corespondență nume noduri la indici $1..V$**
 1. In structura care descrie nodul se ține minte și numele nodului și indicele
 2. Se mapează numele nodului în întregi de la $1..V$ și definesc două funcții
 - `int index_node(char *nume)`
 - `char* nume(int index_node)`
 3. Se realizează o structură separată în care se mapează numele nodului la indice

Reprezentarea prin matrice de adiacență

- Matrice de 2 dimensiuni $a[\text{maxV}][\text{maxV}]$

$a[x][y] = 1$ dacă există arc de la x la y

$a[x][y] = 0$ dacă nu există arc de la x la y

$a[x][y] = \text{cost_arc} (>0)$ dacă există arc de la x la y

$a[x][y] = 0$ dacă nu există arc de la x la y

- Dacă graful este neorientat matricea este simetrică

Reprezentarea prin matrice de adiacență

```
typedef struct
{
    int    nn;    /* numar noduri */
    int    Ma[maxV][maxV];
} TGraphM;
```

```
typedef struct /* matrice alocata dinamic */
{
    int    nn;    /* numar noduri */
    int    **Ma;
} TGraphM1;
```

```
typedef struct /* vector alocat dinamic */
{
    int    nn;    /* numar noduri */
    TCost  *Ma;
} TGraphM2;
```


Citire graf neorientat - matrice de adiacență

```
scanf("%d %d\n", &V, &E);
gm.nn=V;
for(i=1;i<=V;i++)
    for(j=1;j<=V;j++) gm.Ma[i][j]=0;
for(i=1;i<=E;i++)
{
    scanf("%c %c\n", &v1, &v2);
    x = index_node(v1);
    y = index_node(v2);
    gm.Ma[x][y]=1;
    gm.Ma[y][x]=1; /* daca graf
                    neorientat */
}
```

Reprezentarea prin listă de adiacență

```
typedef struct node
{
    int v;      /*indici nod destinatie ai unui arc */
    TCost c;    /* costul asociat arcului */
    struct node *next;
} TNode, *ATNode;
```

```
typedef struct
{
    int nn; /* numar noduri in graf */
    ATNode adl[maxV];
} TGraphL;
```

```
typedef struct
/* alternativ, aloc dinamic vectorul de pointeri */
{
    int nn;
    ATNode *adl;
} TGraphL1;
```

Citire graf - listă de adiacență

```
scanf("%d %d\n", &V, &E);
g.nn=V;
for(i=1;i<=V;i++) g.adl[i]=NULL;
printf("\n");
for(j=1;j<=E;j++)
{   scanf("%c %c\n", &v1, &v2);
    x = index_node(v1);
    y = index_node(v2);
    t=(ATNode)malloc(sizeof(TNode));
    t->v = y; t->next = g.adl[x]; g.adl[x]=t;
/* si daca graf neorientat */
    t=(ATNode)malloc(sizeof(TNode));
    t->v = x; t->next = g.adl[y]; g.adl[y]=t;
}
```



Reprezentarea prin listă de adiacență

- Ordinea în care arcele apar la intrare este importată deoarece determină ordinea în care acestea apar în lista de adiacență.
- Un același graf poate fi reprezentat în diferite moduri prin liste de adiacență.

6. Parcurgerea grafurilor

- Metodă de vizitare sistematică a fiecărui nod și arc
- Parcurgerea grafului poate rezolva multe probleme asociate grafurilor: conectivitate, componente conexe, dacă conține cicluri
- Vector **val[k]** corespunzător nodurilor din graf
 - **val[k] = 0** nod nevizitat
 - **val[k] = ind** nod vizitat
- **Parcurgere în adâncime**
- **Parcurgere pe nivel sau pe lățime**

Parcurgerea în adâncime recursiv

Algoritm DFS recursiv

Inițializează ind la 0

visit(nod **v**)

val[v] = ++ind

pentru fiecare nod **u** legat direct la **v** **executa**

daca val[u] = 0 **atunci** visit(u)

sfarsit

Parcurgerea în adâncime recursiv

dfs()

pentru fiecare nod **k** **executa** $val[k] = 0$

pentru fiecare nod **v** **executa**

daca $val[k] = 0$ /* incepe o noua comp conexa */

atunci **visit**(k)

sfarsit

Parcurgerea în adâncime - LA

```
TGraphL g;
int val[maxV];

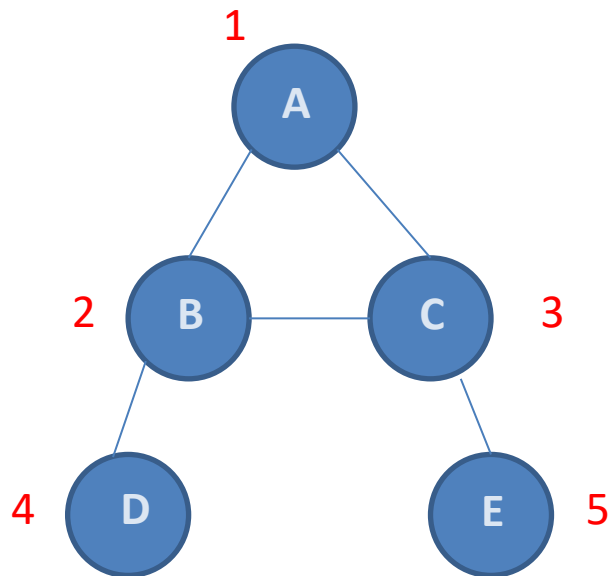
void visit(int k)
{
    ATNode t;
    val[k] = ++ind;
    for(t = g.adl[k]; t != NULL; t = t->next)
        if(val[t->v]==0) visit(t->v);
}

void dfs()
{
    int i;
    for(i=1; i<=V; i++) val[i]=0;
    for(i=1; i<=V; i++)
        if(val[i]==0)
        {
            printf("\nComponenta conexa \n");
            visit(i);
        }
}
```


Parcurgerea în adâncime

- **Parcurgerea în adâncime** a unui graf cu V noduri și E arce reprezentat prin **listă de adiacență** are un timp de execuție de ordinul $O(V+E)$
- **Parcurgerea în adâncime** a unui graf cu V noduri și E arce reprezentat prin **matrice de adiacență** are un timp de execuție de ordinul $O(V^2)$

Depth first recursiv



A	1	2	3	
B	2	4	3	1
C	3	5	2	1
D	4	2		
E	5	3		

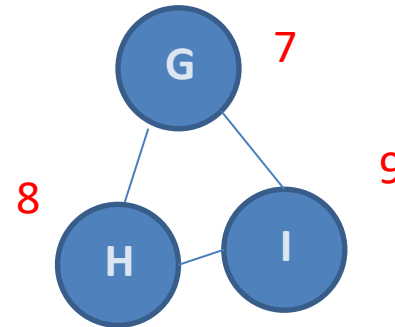
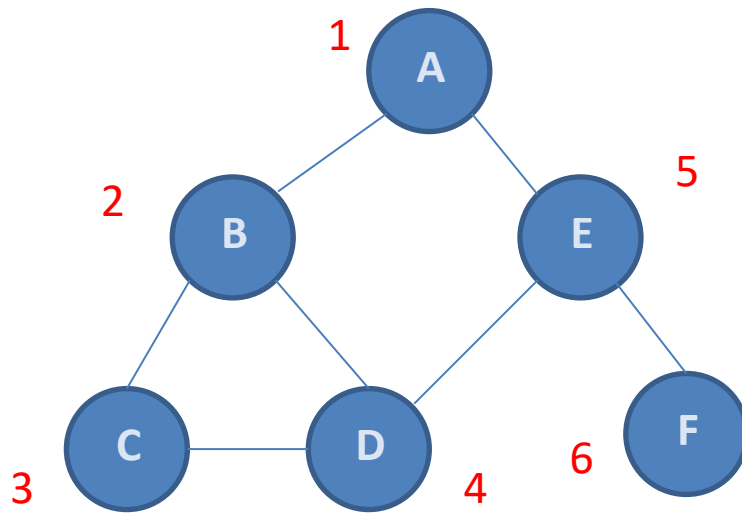
Ordinea de parcurgere
A B D C E

```
Introduceti numar noduri si arce
5 5
A C
A B
B C
C E
B D
a
Noduri adiacente nodului 1 : 2 3
Noduri adiacente nodului 2 : 4 3 1
Noduri adiacente nodului 3 : 5 2 1
Noduri adiacente nodului 4 : 2
Noduri adiacente nodului 5 : 3

Ordinea de parcurgere

Componenta conexa
A B D C E
Process returned 5 (0x5)    execution time :
```

Depth first recursiv



Ordinea de parcurgere

A B C D E F

G H I

A	1	2	5
B	2	3	4 1
C	3	4	2
D	4	5	3 2
E	5	4	6 1
F	6	5	
G	7	8	9
H	8	9	7
I	9	8	7

Introduceti numar noduri si arce

9 10

A E

A B

B D

B C

C D

E F

E D

G I

G H

H I

a

Noduri adiacente nodului 1 : 2 5

Noduri adiacente nodului 2 : 3 4 1

Noduri adiacente nodului 3 : 4 2

Noduri adiacente nodului 4 : 5 3 2

Noduri adiacente nodului 5 : 4 6 1

Noduri adiacente nodului 6 : 5

Noduri adiacente nodului 7 : 8 9

Noduri adiacente nodului 8 : 9 7

Noduri adiacente nodului 9 : 8 7

Ordinea de parcurgere

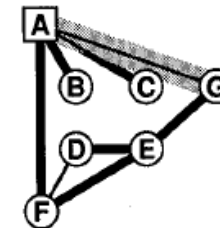
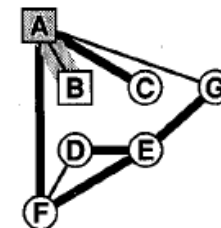
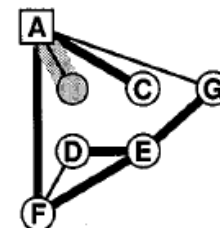
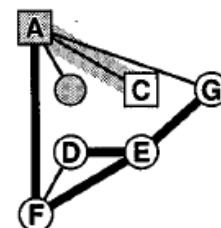
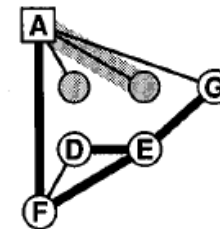
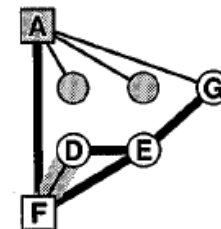
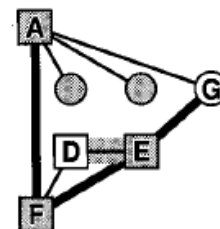
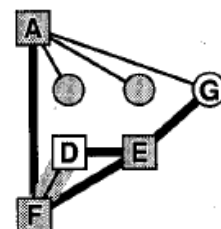
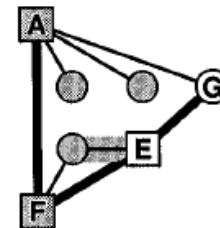
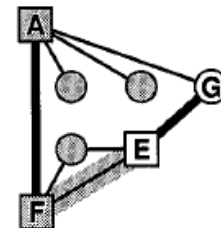
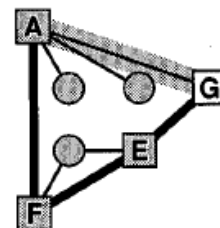
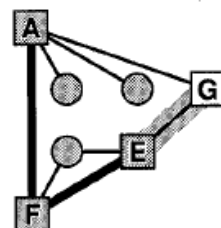
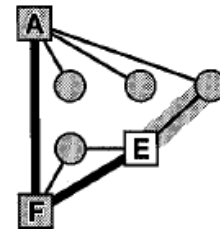
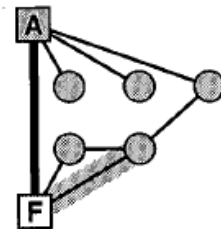
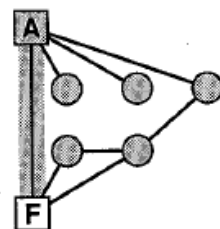
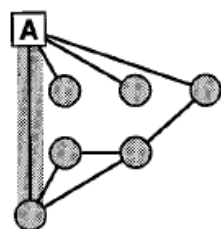
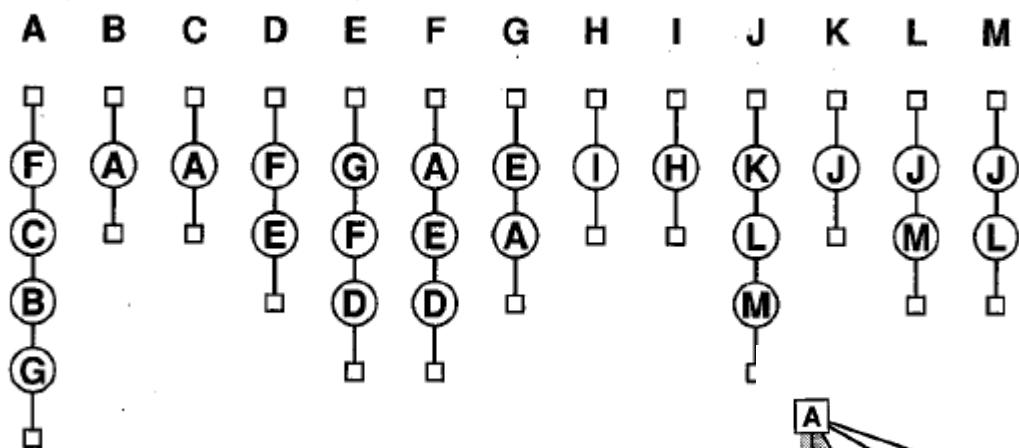
Componenta conexa

A B C D E F

Componenta conexa

G H I

Process returned 9 (0x9) execution time



Parcurgerea în adâncime nerecursiv

- Se folosește o stivă
- **val[k] = 0** nod nedescoperit
- **val[k] = -1** nod în stivă (în curs de vizitare)
- **val[k] = +1** sau **index vizitare** pt nod vizitat

Parcurgerea în adâncime nerecursiv

Algoritm DFS nerecursiv

visitNR(nod v)

push(v)

cat timp !empty_stack **repeta**

v = pop()

val[v] = ++ind

pentru fiecare nod u legat direct la v **executa**

daca val[u] = 0

atunci

push(v)

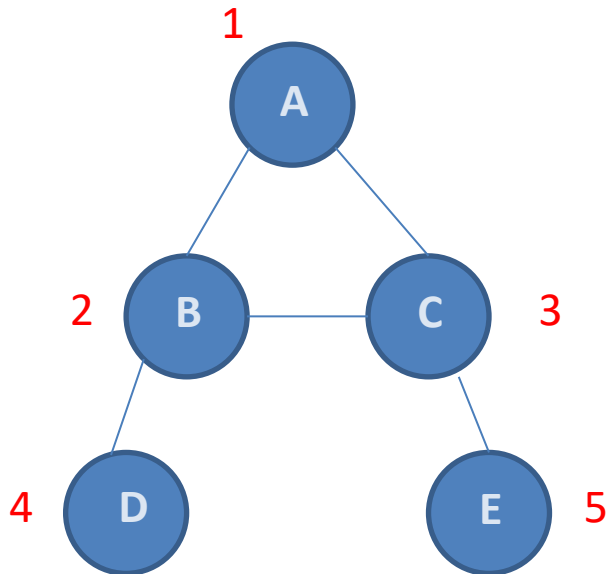
val[v] = -1

sfarsit

Parcurgerea în adâncime nerecursiv

```
void visitNR(int k)
{
    ATNode t;
    push(s, k);
    while (!stackempty(s))
    {
        k = pop(s); val[k] = ++ind;
        printf("%d ", k);
        for(t = g.adl[k]; t!=NULL; t=t->next)
            if(val[t->v] == 0)
            {
                push(s, t->v);
                val[t->v] = -1;
            }
    }
}
```

Depth first nerecursiv

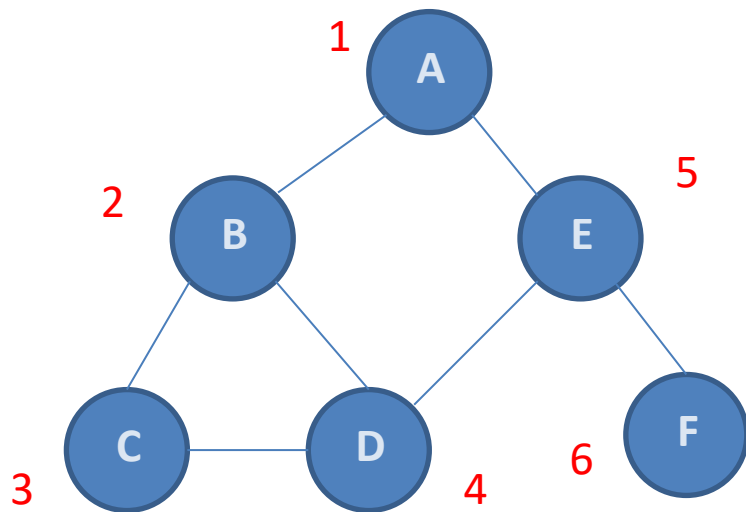


A 1 2 3
B 2 4 3 1
C 3 5 2 1
D 4 2
E 5 3

Ordinea de parcurgere
A C E B D

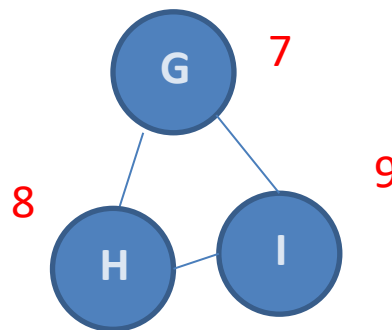
```
Introduceti numar noduri si arce
5 5
A C
A B
B C
C E
B D
a
Noduri adiacente nodului 1 : 2 3
Noduri adiacente nodului 2 : 4 3 1
Noduri adiacente nodului 3 : 5 2 1
Noduri adiacente nodului 4 : 2
Noduri adiacente nodului 5 : 3

Ordinea de parcurgere
Componenta conexa
A C E B D
Process returned 5 (0x5)    execution time
```

A	1	2	5	
B	2	3	4	1
C	3	4	2	
D	4	5	3	2
E	5	4	6	1
F	6	5		
G	7	8	9	
H	8	9	7	
I	9	8	7	

Depth first nerecursiv



Ordinea de parcurgere

A E F D C B

G I H

Introduceti numar noduri si arce

9 10

A E

A B

B D

B C

C D

E F

E D

G I

G H

H I

a

Noduri adiacente nodului 1 : 2 5

Noduri adiacente nodului 2 : 3 4 1

Noduri adiacente nodului 3 : 4 2

Noduri adiacente nodului 4 : 5 3 2

Noduri adiacente nodului 5 : 4 6 1

Noduri adiacente nodului 6 : 5

Noduri adiacente nodului 7 : 8 9

Noduri adiacente nodului 8 : 9 7

Noduri adiacente nodului 9 : 8 7

Ordinea de parcurgere

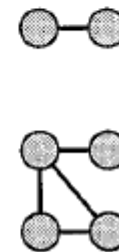
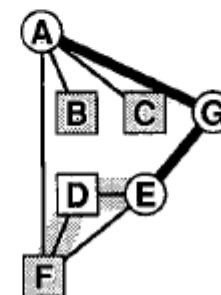
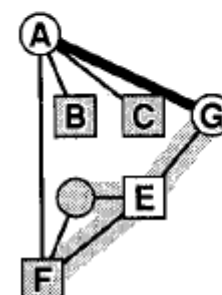
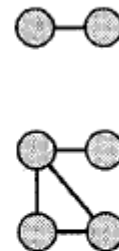
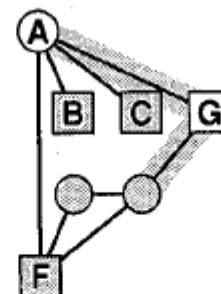
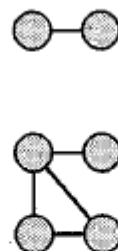
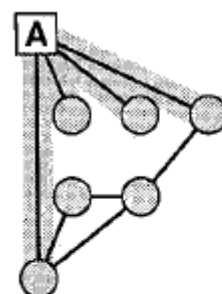
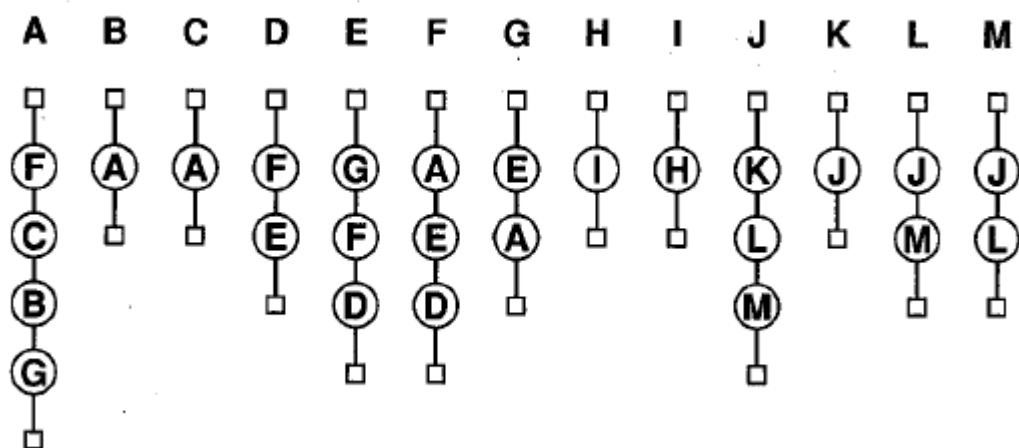
Componenta conexa

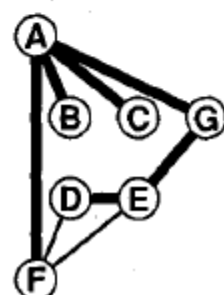
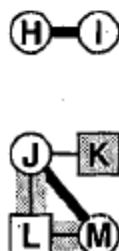
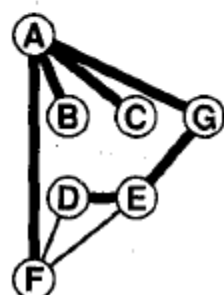
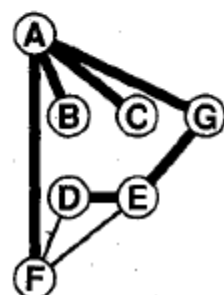
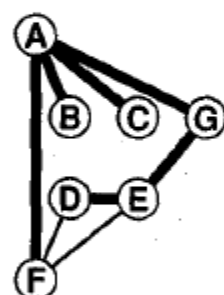
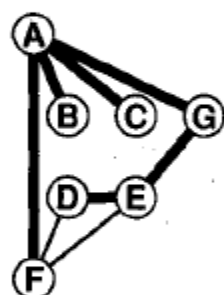
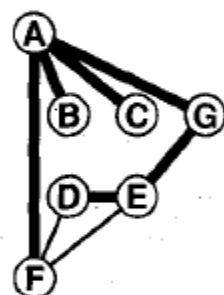
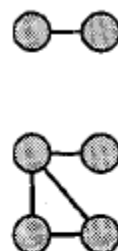
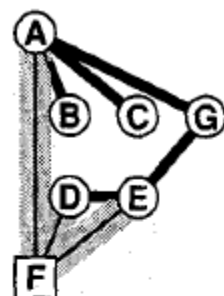
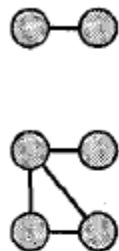
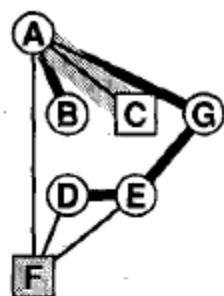
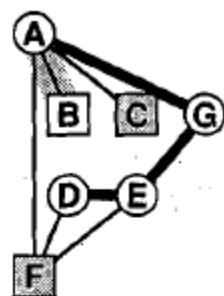
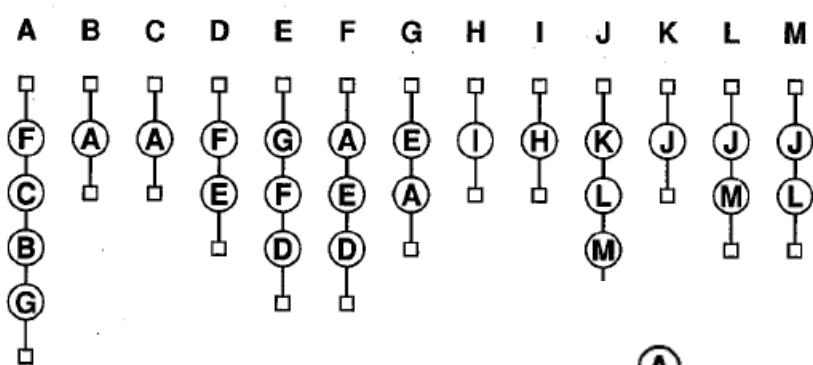
A E F D C B

Componenta conexa

G I H

Process returned 9 (0x9) execution time :





Parcurgerea pe nivel

Algoritm BFS nerecursiv

visitBFS(nod v)

enqueue(v)

cat timp !queue_stack **repeta**

v = dequeue()

val[v] = ++ind

pentru fiecare nod u legat direct la v **executa**

daca val[u] = 0

atunci

enqueue(v)

val[v] = -1

sfarsit

Parcurgerea pe nivel

bfs()

pentru fiecare nod **k** **executa** $val[k] = 0$

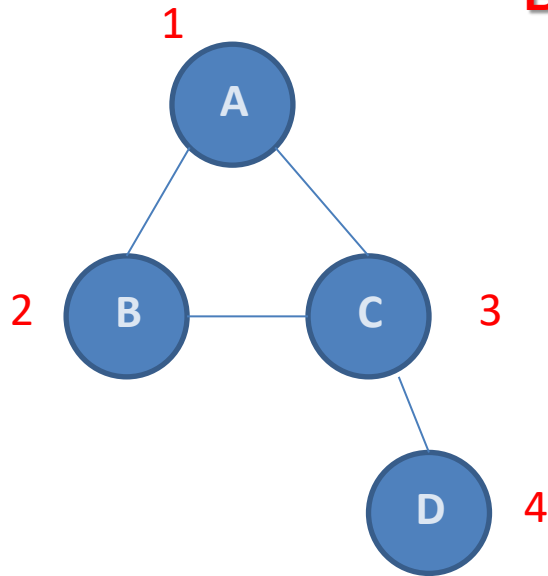
pentru fiecare nod **v** **executa**

daca $val[k] = 0$ /* incepe o noua comp conexa */

atunci **visitBFS**(k)

sfarsit

Breadth first



A	1	2	3	
B	2	3	1	
C	3	4	2	1
D	4	3		

Ordinea de parcurgere

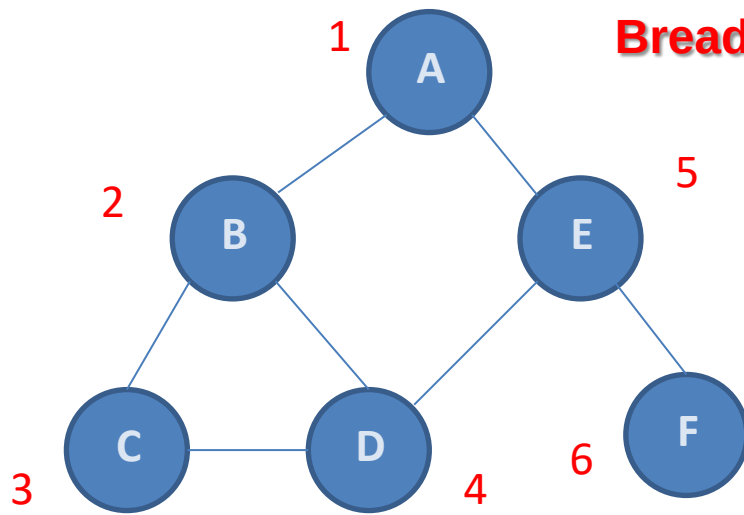
A B C D

```
Introduceti numar noduri si arce
4 4
A C
A B
B C
C D
a
Noduri adiacente nodului 1 : 2 3
Noduri adiacente nodului 2 : 3 1
Noduri adiacente nodului 3 : 4 2 1
Noduri adiacente nodului 4 : 3

Ordinea de parcurgere

Componenta conexa
A B C D
Process returned 4 (0x4)    execution time
```

Breadth first



Ordinea de parcurgere

A B E C D F
G H I

A	1	2	5	
B	2	3	4	1
C	3	4	2	
D	4	5	3	2
E	5	4	6	1
F	6	5		
G	7	8	9	
H	8	9	7	
I	9	8	7	

```
Introduceti numar noduri si arce
9 10
A E
A B
B D
B C
C D
E F
E D
G I
G H
H I
a
Noduri adiacente nodului 1 : 2 5
Noduri adiacente nodului 2 : 3 4 1
Noduri adiacente nodului 3 : 4 2
Noduri adiacente nodului 4 : 5 3 2
Noduri adiacente nodului 5 : 4 6 1
Noduri adiacente nodului 6 : 5
Noduri adiacente nodului 7 : 8 9
Noduri adiacente nodului 8 : 9 7
Noduri adiacente nodului 9 : 8 7

Ordinea de parcurgere

Componenta conexa
A B E C D F
Componenta conexa
G H I
Process returned 9 (0x9)    execution t
```